

# Interrogation Testing of CHC Solvers

DAVID KAINDLSTORFER, TU Wien, Austria

ANASTASIA ISYCHEV, TU Wien, Austria

VALENTIN WÜSTHOLZ, Diligence Security, Austria

MARIA CHRISTAKIS, TU Wien, Austria

A Constrained Horn Clause (CHC) is a specific type of logic formula that contains uninterpreted predicates. CHC formulas are often used by static program analyzers to encode program properties, which are then verified using CHC solvers. The solvers themselves are complex tools and may contain bugs, which can lead to verifying unsafe programs, flagging safe programs as unsafe, or providing analyzers with incorrect invariants and counterexamples. It is, therefore, crucial to develop techniques for systematically testing CHC solvers.

In this paper, we present the first interrogation-testing technique for CHC solvers, which we implement in a tool called HORNGATOR. Our technique uses witnesses generated by the solver under test to form new CHC instances. It also integrates a knowledge base maintaining a history of past solver queries. All this information helps HORNGATOR generate more diverse instances, thereby improving its bug-finding effectiveness. As a result, HORNGATOR found 21 unique bugs in five state-of-the-art CHC solvers, all of which are confirmed by the developers, 18 are fixed, and eight are of the highest severity.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**.

Additional Key Words and Phrases: interrogation testing, CHC solvers, fuzzing

## ACM Reference Format:

David Kaundlstorfer, Anastasia Isychev, Valentin Wüstholtz, and Maria Christakis. 2026. Interrogation Testing of CHC Solvers. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE076 (July 2026), 20 pages. <https://doi.org/10.1145/3797104>

## 1 Introduction

A *Constrained Horn Clause* (CHC) is a specific type of logic formula that includes *uninterpreted* (i.e., unknown) predicates. A CHC *instance* is a set of CHCs; given an instance as input, a CHC solver tries to instantiate all uninterpreted predicates such that the set of CHCs is satisfiable. Examples of popular CHC solvers are SPACER [22] and ELDARICA [16]. Such solvers are widely used in static program analysis and verification [3], for instance, in SEAHORN [15] for LLVM bitcode, JAYHORN [17] for Java, and RUSTHORN [26] for Rust.

**Bugs in CHC solvers.** CHC solvers are complex tools, which makes it quite likely that they contain bugs in their implementations. Tab. 1 shows the severity of the different types of bugs that may occur in CHC solvers. For bugs of severity 1 (the highest severity), the solver returns SAT (i.e., satisfiable) for an instance that is UNSAT (i.e., unsatisfiable). Such bugs may have detrimental consequences; they may render the program analyzers that rely on the CHC solver unsound. As a result, unsafe code could be proved safe by the affected analyzers.

Bugs of severity 2 cause the solver to return UNSAT for a SAT instance. This type of bugs may result in incompleteness in program analyzers. In other words, the affected analyzers could flag safe

---

Authors' Contact Information: David Kaundlstorfer, TU Wien, Vienna, Austria, david.kaundlstorfer@tuwien.ac.at; Anastasia Isychev, TU Wien, Vienna, Austria, anastasia.isychev@tuwien.ac.at; Valentin Wüstholtz, Diligence Security, Vienna, Austria, valentin.wustholtz@diligence.security; Maria Christakis, TU Wien, Vienna, Austria, maria.christakis@tuwien.ac.at.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE076

<https://doi.org/10.1145/3797104>

Tab. 1. Severity of the different types of bugs that may occur in CHC solvers. GT stands for ground truth and SR for solver response.

| GT \ SR | SAT | UNSAT | UNKNOWN | Other |
|---------|-----|-------|---------|-------|
|         | SAT | UNSAT | UNKNOWN | Other |
| SAT     | 3a  | 2     | 4a      | 4b    |
| UNSAT   | 1   | 3b    | 4a      | 4b    |

code as unsafe. This can, in turn, lead to developers wasting valuable time looking for a non-existent bug in their code.

Bugs can also occur when the solver returns the correct response; see severity 3 in the table. Consider that when an instance is found to be SAT, the solver can typically provide a *model*, that is, an instantiation of all uninterpreted predicates in the instance such that it is satisfiable. However, the model could be incorrect due to a solver bug (severity 3a); in particular, the instantiation of certain predicates might not lead to a satisfiable instance. As a consequence, program analyzers that use CHC solvers for invariant inference (for instance, by using uninterpreted predicates as “placeholders” when capturing the constraints that an inductive loop invariant would have to satisfy) could infer incorrect invariants.

On the other hand, when an instance is found to be UNSAT, the solver typically provides a *refutation proof*, which can be used to construct a counterexample trace. Just like models in the SAT case, refutation proofs may also be incorrect (severity 3b). This could lead to analyzers generating wrong counterexamples for unsafe code, potentially misleading developers when debugging.

Finally, severity 4a refers to cases where the solver returns UNKNOWN for instances that lie in a decidable theory fragment, and severity 4b to solver crashes, insufficient input validation, and other less critical bugs.

**Related work.** Given the consequences of the more severe bug types, it is crucial to develop techniques for systematically testing CHC solvers.

*Why not use differential testing [27]?* Differential testing, although widely used, is ill-suited as the primary approach for CHC solvers. These tools differ widely in the theories they support—e.g., only some handle arrays or algebraic data types (ADTs)—as well as in their dialects and accepted syntax. For instance, bug 1 in Sect. 5 involves ADTs unsupported by CHC solvers GOLEM [4], THETA [33], and ULTIMATE TREE AUTOMIZER [13], and bug 2 relies on a solver-specific construct that only ELDARICA understands. In addition, certain high-complexity instances can only be solved by a single tool while others fail to respond; for example, bug 5 can only be handled by ELDARICA. These discrepancies make consistent comparisons across solvers infeasible. More generally, even when solvers can process the same input, divergent answers do not reveal which solver is at fault, undermining the diagnostic value of differential testing.

*How about SMT-solver fuzzers (e.g., [20, 24, 30, 34])?* While SMT and CHC solvers share a common input language (SMT-LIB [2]), they solve fundamentally different problems: SMT solvers decide the satisfiability of formulas, whereas CHC solvers synthesize interpretations for uninterpreted predicates that satisfy a system of Horn clauses. For instance, invariant synthesis—a core verification task—can be reduced to CHC solving, but not to SMT solving. This distinction introduces fundamental challenges for reusing SMT fuzzers, as CHC solvers differ not only in the types of inputs they consume but also in the outputs they produce.

In terms of inputs, although both classes of solvers support SMT-LIB, they do so in mutually incompatible ways. SMT fuzzers typically generate formulas without uninterpreted predicates, making them inadequate for exercising CHC-specific logic. As an analogy, SMT fuzzers are to CHC fuzzers what C-compiler fuzzers are to Java-compiler fuzzers: some Java code may also be valid C,

but effectively testing Java requires handling different syntax and semantics. Similarly, SMT-level mutations often violate the structural constraints imposed by CHC solvers, leading to syntactically invalid inputs.

In terms of outputs, CHC solvers may provide predicate models or refutation proofs in addition to SAT/UNSAT. SMT solvers also return models, but these typically assign concrete values to first-order variables; likewise, when available, SMT proof objects differ substantially in structure from CHC refutation proofs. The CHC-specific outputs are central to HORN GATOR's design as they drive its transformations and are essential for bug finding. Moreover, SMT fuzzers tend to focus on refutational soundness bugs—returning UNSAT for satisfiable inputs—which are considered the most severe from a program-analysis perspective. In contrast, the most critical bugs in CHC solvers involve incorrectly returning SAT for unsatisfiable systems, a class of errors that certain SMT fuzzers cannot detect (e.g., [24]).

Given these limitations, there is a clear need for dedicated techniques to test CHC solvers that do not rely on differential testing. Recent work proposed HORN FUZZ [31], a technique based on *metamorphic testing* [12] that is able to detect bugs of severity 1, 2, and 3a. HORN FUZZ was used to test SPACER; it detected two bugs of severity 2 and 13 bugs of severity 3a, where the generated model was incorrect. However, as we will see, HORN FUZZ relies primarily on SMT-style rewrites (e.g., reordering conjuncts) and fails to uncover several severe bugs. To the best of our knowledge, there has been no other dedicated attempt to test CHC solvers.

**Our approach.** In this paper, we present the first interrogation-testing technique for CHC solvers. *Interrogation testing* [18] is a metamorphic-testing approach that can be applied to a wide range of analysis tools. Its core principles—leveraging analysis witnesses and maintaining a knowledge base—are domain independent. Witnesses, such as inferred invariants or proofs, provide information beyond the basic analysis response and can be systematically used to generate new, semantically related analysis queries. The knowledge base serves as a persistent repository of queries, responses, and associated witnesses, enabling the tester to generate increasingly diverse and targeted test cases. In prior work, interrogation testing was applied to program analyzers that reason about reachability.

We instantiate interrogation testing for CHC solvers for the first time. This requires designing CHC-specific transformations, as CHC solvers produce fundamentally different artifacts—namely, models for predicates and refutation proofs—compared to the reachability witnesses used in prior work. Our instantiation leverages both types of solver feedback—models and proofs—going beyond simple SAT or UNSAT responses, and uses the knowledge base to generate new CHC instances with known satisfiability. These instances are both semantically meaningful and syntactically diverse, making them more effective for exposing bugs in CHC solvers.

We implement our technique in a tool called HORN GATOR and use it to test five CHC solvers that participate in CHC-COMP [1], namely ELDARICA [16], GOLEM [4], SPACER [22], THETA [33], and ULTIMATE TREE AUTOMIZER [13]. Our tool focuses on detecting bugs of severity 1–3; bugs of lower severity are less critical and easy to find. In particular, bugs of severity 4a can be found by passing instances of decidable theory fragments (e.g., from CHC-COMP) to the solver and reporting a bug whenever the solver returns UNKNOWN. Bugs of severity 4b can be found by generating random instances, even non-CHC ones, and passing them to the solver.

HORN GATOR found a total of 21 unique bugs in all solvers we tested. All bugs were confirmed by the developers, and 18 of them are already fixed. Eight bugs have severity 1, ten have severity 2, and the remaining three have severity 3. As shown by our experimental evaluation, our bugs would not have been found or would have been found with a significant slowdown without the two core principles of interrogation testing. We also show that HORN GATOR is significantly more effective in bug finding than HORN FUZZ.

**Our contributions.** Our paper makes the following contributions:

- We present an interrogation-testing technique for detecting severe bugs in CHC solvers.
- We implement our technique in the open-source tool HORNGATOR.
- We evaluate the effectiveness of HORNGATOR by testing five solvers that participate in CHC-COMP. Our results significantly push the state of the art.

**Outline.** Next, we provide brief background on CHCs. In Sect. 3, we give an overview of HORNGATOR, and in Sect. 4, we explain the technical details of our approach. We present our experimental evaluation in Sect. 5, discuss related work in Sect. 6, and conclude in Sect. 7.

## 2 Background: Constrained Horn Clauses

A CHC *instance* is a set of CHCs. Each CHC is a first-order logic formula of the following form [14]:

$$\forall V \cdot \varphi \wedge p_1(X_1) \wedge \dots \wedge p_n(X_n) \rightarrow h(X)$$

Here,  $V$  denotes a set of variables and  $X_1, \dots, X_n, X$  are terms over  $V$ .  $\varphi$  denotes a constraint over a background theory, such as linear arithmetic, arrays, bit-vectors, etc.,  $p_1, \dots, p_n$  are uninterpreted predicates, and  $h$  is an uninterpreted predicate or  $\perp$  (i.e., *false*). Without loss of generality, let us assume that  $\varphi$  is a conjunction of sub-constraints  $\varphi_1 \wedge \dots \wedge \varphi_m$ . Note that this conjunction may express any  $\varphi$ .

An instance is *satisfiable* if there exists at least one instantiation of all uninterpreted predicates in the instance such that the set of CHCs holds. As an example, consider the following CHC instance:

$$\forall x \cdot x \leq 0 \rightarrow P(x) \tag{1}$$

$$\forall x, y \cdot P(x) \wedge x < 2 \wedge y = x + 1 \rightarrow P(y) \tag{2}$$

$$\forall x \cdot P(x) \wedge x \geq 3 \rightarrow \perp \tag{3}$$

This set of clauses is satisfiable, and a model for the uninterpreted predicate is:  $P(x) = \{x \mid x < 3\}$ .

An instance is *unsatisfiable* if there is no instantiation of the uninterpreted predicates in the instance such that the set of CHCs holds. For example, when changing clause (3) to

$$\forall x \cdot P(x) \wedge x \geq 2 \rightarrow \perp \tag{4}$$

the set of clauses becomes unsatisfiable. The refutation proof derives  $\perp$  as follows:

$$\top \xrightarrow{(1)} P(0) \xrightarrow{(2)} P(1) \xrightarrow{(2)} P(2) \xrightarrow{(3)} \perp$$

It says that, if  $x = 0$ , we know that  $P(0)$  holds from clause (1). Given  $P(0)$ , the left-hand side of clause (2) holds ( $P(0) \wedge 0 < 2 \wedge y = 0 + 1$ ), and thus, clause (2) establishes  $P(1)$  on the right-hand side. Given  $P(1)$ , clause (2) can now also establish  $P(2)$ , i.e.,  $P(1) \wedge 1 < 2 \wedge y = 1 + 1 \rightarrow P(2)$ . In turn,  $P(2)$  makes the left-hand side of clause (4) hold, i.e.,  $P(2) \wedge 2 \geq 2$ , which derives  $\perp$ . In this proof,  $P(0)$ ,  $P(1)$ , and  $P(2)$  are called *proof facts* [4].

For simplicity, we omit the outermost universal quantifier in the rest of this paper, and we consider all free variables to be universally quantified.

## 3 Overview

Fig. 1 shows an overview of HORNGATOR. Given a CHC *seed instance*—our implementation supports multiple seed instances—and a CHC solver under test, the *interrogator* component initially queries the solver with the seed instance. The seed instance could be taken from CHC-COMP, the solver’s test suite, or it may be provided by the user. In our experiments, we use instances from CHC-COMP.

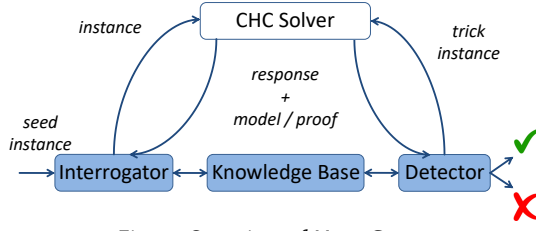


Fig. 1. Overview of HORNGATOR.

When querying the solver with an instance, the interrogator requests not only the solver response (e.g., SAT or UNSAT), but also the model in case of SAT and the refutation proof in case of UNSAT. Let us refer to the model or refutation proof as the *witness*.

If the solver does not timeout or return UNKNOWN, then the CHC instance is suitable for detecting solver bugs of severity 1–3. In this case, the instance together with the solver response and the corresponding witness are stored in the *knowledge base* (partially inspired by how greybox fuzzers use a corpus of interesting inputs). Such an interrogation round may be repeated any number of times to gradually enrich the knowledge base. The interrogator may also use the knowledge base to construct new CHC instances (by mutating existing ones) with which to query the solver.

At any point, the *detector* component may query the solver with a *trick instance*. A trick instance is a CHC instance constructed from information in the knowledge base such that the expected solver response is known a-priori. In other words, the detector uses one or more instances from the knowledge base together with the corresponding solver response and witness to generate a new instance whose satisfiability is known in advance. In Sect. 4, we introduce different categories of transformations that HORNGATOR applies to derive trick queries. The transformations, and especially how they use the response and witness, are one of the key novelties of our approach.

If the solver response for the trick instance contradicts the expected response, HORNGATOR reports a bug. The bug severity is determined through manual inspection as we discuss later. If no bug is detected and the trick instance is suitable for further testing (i.e., no UNKNOWN response or timeout), the instance and all relevant information are added to the knowledge base.

**Example bug.** Starting from an existing instance from CHC-COMP’23 (in the “linear integer arithmetic with nonlinear clauses” category), HORNGATOR detected a bug of severity 1 in the GOLEM solver (bug 9 in Tab. 2). In particular, the solver returned SAT for the following unsatisfiable instance:

$$\begin{aligned}
 x < y &\rightarrow P(x, y) \\
 P(x, 1) &\rightarrow Q \\
 Q \wedge P(x, 7) \wedge x > y &\rightarrow R(y) \\
 R(3) &\rightarrow \perp
 \end{aligned}$$

Note that the original instance from CHC-COMP is unsatisfiable and contains almost 18K LOC. The detector eventually reports the above instance (minimized both automatically and manually for presentation purposes) after repeatedly querying the solver with instances generated using the knowledge base. To find this buggy instance, HORNGATOR applied transformations that replace conjuncts on the left-hand side of a clause with  $\top$  (i.e., *true*); as we explain later, when applied to an UNSAT instance, such a transformation preserves its satisfiability (i.e., the transformed instance should also be UNSAT). In addition, HORNGATOR dropped clauses that do not appear in the refutation proof, and therefore, are not needed by the solver to derive UNSAT.

This bug was quickly fixed by the Golem developers, who thanked us and encouraged us to send them more such issues.

## 4 Transformations

In this section, we describe in detail the transformations that the interrogator and detector may apply to instances stored in the knowledge base to generate new ones. Note that the interrogator can apply any transformation to any instance (regardless of its satisfiability); its goal is to enrich the knowledge base with diverse instances, which is crucial for exploring different solver components and optimization paths. The detector, on the other hand, carefully applies transformations such that the solver response for the transformed instances can be anticipated and a bug can be detected in case of a contradiction.

The detector categorizes the transformations as follows:

- **SAT-to-SAT.** These transformations are applied to a satisfiable instance and ensure that the transformed instance is also satisfiable.
- **UNSAT-to-UNSAT.** These transformations are applied to an unsatisfiable instance and ensure that the transformed instance is also unsatisfiable.
- **Satisfiability preserving.** These transformations are applied to any instance, satisfiable or unsatisfiable, and ensure that the transformed instance preserves the satisfiability of the original instance. In particular, if the original instance is satisfiable (resp. unsatisfiable), the transformed instance is also satisfiable (resp. unsatisfiable).
- **Instance fusion.** These transformations fuse two instances into one, and the satisfiability of the newly generated instance depends on the satisfiability of the original instances.

### 4.1 SAT-to-SAT Transformations

We have designed the following five transformations for this category.

**PLUGMODEL LHS.** Given a SAT instance, this transformation first chooses a clause  $\varphi \wedge p_1 \wedge \dots \wedge p_n \rightarrow h$  and a left-hand-side (LHS) uninterpreted predicate  $p_i$ , where  $i \in \{1, \dots, n\}$ . It then replaces  $p_i$  with its instantiation from the model, denoted by  $M(p_i)$ , obtaining

$$\varphi \wedge \mathbf{M}(p_i) \wedge p_1 \wedge \dots \wedge p_{i-1} \wedge p_{i+1} \wedge \dots \wedge p_n \rightarrow h.$$

Since the original instance is SAT, there must exist a model  $M$  that provides instantiations for all uninterpreted predicates such that the CHCs hold. Intuitively,  $M$  is still a model for any transformed instance that instantiates *some* of the uninterpreted predicates. Consequently, when replacing an uninterpreted predicate with one of these instantiations—specifically, from the model generated by the CHC solver—the transformed instance should remain satisfiable.

As an example, consider the clause

$$x \leq 5 \wedge \mathbf{P}(\mathbf{x}, \mathbf{y}) \wedge Q(y) \rightarrow R(x),$$

where  $M(P) = x \geq y$ . When applying this transformation, we obtain the clause

$$x \leq 5 \wedge \mathbf{x} \geq \mathbf{y} \wedge Q(y) \rightarrow R(x).$$

Note that in this and all following transformations, we highlight how the formulas are transformed in bold.

**PLUGMODEL RHS.** Given a SAT instance, this transformation chooses a clause  $\varphi \wedge p_1 \wedge \dots \wedge p_n \rightarrow h$  and replaces the right-hand-side (RHS) uninterpreted predicate  $h$  with its instantiation from the model, denoted by  $M(h)$ . To preserve the CHC syntax, we rewrite the clause to

$$\varphi \wedge \neg \mathbf{M}(h) \wedge p_1 \wedge \dots \wedge p_n \rightarrow \perp.$$

The above clause is equivalent to the implication

$$\varphi \wedge p_1 \wedge \dots \wedge p_n \rightarrow M(h),$$

which however does not adhere to the CHC syntax. They are equivalent because of the implication property

$$(A \rightarrow B) \equiv (A \wedge \neg B \rightarrow \text{false}).$$

As an example, consider the clause

$$x \leq 5 \wedge P(x, y) \wedge Q(y) \rightarrow R(x),$$

where  $M(R) = x \geq 0$ . When applying this transformation, we obtain the clause

$$x \leq 5 \wedge \neg(x \geq 0) \wedge P(x, y) \wedge Q(y) \rightarrow \perp.$$

**ADDCTRLHS.** Given a SAT instance, this transformation chooses a clause  $\varphi \wedge p_1 \wedge \dots \wedge p_n \rightarrow h$  and conjoins any constraint  $\varphi_{m+1}$  to  $\varphi \equiv \varphi_1 \wedge \dots \wedge \varphi_m$ , potentially including  $\perp$ . Note that  $\varphi_{m+1}$  needs to be over the same background theory as  $\varphi$ .

The transformation strengthens the LHS of a satisfiable implication  $(A \rightarrow B) \equiv (\neg A \vee B)$ . Since the original disjunction  $\neg A \vee B$  holds, the transformed disjunction  $\neg(A \wedge \varphi_{m+1}) \vee B = \neg\varphi_{m+1} \vee (\neg A \vee B)$  should also hold.

As an example consider again the clause

$$x \leq 5 \wedge P(x, y) \wedge Q(y) \rightarrow R(x).$$

A possible transformed clause could be obtained by adding the conjunct  $x \neq y$ :

$$x \leq 5 \wedge x \neq y \wedge P(x, y) \wedge Q(y) \rightarrow R(x)$$

**UNPLUGLHS.** Given a SAT instance, this transformation chooses a conjunct  $\varphi_j$ , where  $j \in \{1, \dots, m\}$ , occurring in  $\varphi \equiv \varphi_1 \wedge \dots \wedge \varphi_m$  of a clause  $\varphi \wedge p_1 \wedge \dots \wedge p_n \rightarrow h$ . It then replaces  $\varphi_j$  with a fresh uninterpreted predicate  $p_{\text{fresh}}$  (of appropriate arity). More specifically, given a clause

$$\varphi_1 \wedge \dots \wedge \varphi_{j-1} \wedge \varphi_j \wedge \varphi_{j+1} \wedge \dots \wedge \varphi_m \wedge p_1 \wedge \dots \wedge p_n \rightarrow h,$$

we obtain

$$\varphi_1 \wedge \dots \wedge \varphi_{j-1} \wedge \varphi_{j+1} \wedge \dots \wedge \varphi_m \wedge p_1 \wedge \dots \wedge p_n \wedge p_{\text{fresh}} \rightarrow h.$$

The intuition here is that the transformed instance should remain satisfiable because there exists a model for  $p_{\text{fresh}}$ , namely  $\varphi_j$  or, trivially,  $\perp$ .

As an example, consider the clause

$$x \geq 5 \rightarrow Q(x),$$

where  $\varphi_j = x \geq 5$ . When applying this transformation, we obtain

$$P(x) \rightarrow Q(x).$$

**UNPLUGRHS.** Given a SAT instance, this transformation chooses a conjunct  $\varphi_j$ , where  $j \in \{1, \dots, m\}$ , occurring in  $\varphi \equiv \varphi_1 \wedge \dots \wedge \varphi_m$  of a clause  $\varphi \wedge p_1 \wedge \dots \wedge p_n \rightarrow \perp$ . Note that the right-hand side is  $\perp$ . It then removes  $\varphi_j$  from the left-hand side and adds a fresh uninterpreted predicate  $p_{\text{fresh}}$  (of appropriate arity) to the right-hand side of the clause. More specifically, given a clause

$$\varphi_1 \wedge \dots \wedge \varphi_{j-1} \wedge \varphi_j \wedge \varphi_{j+1} \wedge \dots \wedge \varphi_m \wedge p_1 \wedge \dots \wedge p_n \rightarrow \perp,$$

we obtain

$$\varphi_1 \wedge \dots \wedge \varphi_{j-1} \wedge \varphi_{j+1} \wedge \dots \wedge \varphi_m \wedge p_1 \wedge \dots \wedge p_n \wedge \rightarrow p_{\text{fresh}}.$$

The intuition here is that the transformed instance should remain satisfiable because there exists a model for  $p_{fresh}$ , namely  $\neg\varphi_j$  or, trivially,  $\top$ . The negation in  $\neg\varphi_j$  is explained by the implication property

$$(A \rightarrow B) \equiv (A \wedge \neg B \rightarrow false).$$

As an example, consider the clause

$$x \neq 5 \wedge \neg(x \geq y) \wedge Q(y) \rightarrow \perp,$$

where  $\varphi_j = \neg(x \geq y)$ . When applying this transformation, we obtain the clause

$$x \neq 5 \wedge Q(y) \rightarrow P(x, y),$$

where a model for  $P$  is  $x \geq y$ .

**Summary.** Transformations `PLUGMODEL`LHS and `PLUGMODEL`RHS transform an instance using both the solver response, i.e., that the original instance is found to be SAT, and the model. The remaining transformations rely only on the solver response.

Transformation `ADDCTRL`LHS essentially strengthens the left-hand side of a satisfiable clause, which as we explained, preserves its satisfiability. The other transformations either instantiate an uninterpreted predicate (`PLUGMODEL`LHS, `PLUGMODEL`RHS) or uninstantiate a constraint (`UNPLUG`LHS, `UNPLUG`RHS).

## 4.2 UNSAT-to-UNSAT Transformations

We have designed the following four transformations for this category.

**PLUGTOP**LHS. Given an UNSAT instance, this transformation chooses a clause  $\varphi \wedge p_1 \wedge \dots \wedge p_n \rightarrow h$  and replaces a conjunct  $\varphi_j$ , where  $j \in \{1, \dots, m\}$ , in  $\varphi \equiv \varphi_1 \wedge \dots \wedge \varphi_m$  or a LHS uninterpreted predicate  $p_i$ , where  $i \in \{1, \dots, n\}$ , with  $\top$ . More specifically, when replacing a conjunct  $\varphi_j$ , we obtain

$$\varphi_1 \wedge \dots \wedge \varphi_{j-1} \wedge \top \wedge \varphi_{j+1} \wedge \dots \wedge \varphi_m \wedge p_1 \wedge \dots \wedge p_n \rightarrow h.$$

Similarly, when replacing a predicate  $p_i$ , we obtain

$$\varphi \wedge p_1 \wedge \dots \wedge p_{i-1} \wedge \top \wedge p_{i+1} \wedge \dots \wedge p_n \rightarrow h.$$

This transformation weakens the LHS of an unsatisfiable implication  $(A \rightarrow B) \equiv (\neg A \vee B)$ . In our case,  $A$  is a conjunction, and thus,  $\neg A$  is a disjunction of the negated conjuncts. By applying our transformation, a disjunct in  $\neg A$  becomes *false*. So, if the original disjunction does not hold, the transformed disjunction should also not hold.

For example, consider the unsatisfiable instance from Sect. 2:

$$\begin{aligned} x \leq 0 &\rightarrow P(x) \\ P(x) \wedge x < 2 \wedge y = x + 1 &\rightarrow P(y) \\ P(x) \wedge x \geq 2 &\rightarrow \perp \end{aligned}$$

When replacing conjunct  $x < 2$  with  $\top$ , we obtain:

$$\begin{aligned} x \leq 0 &\rightarrow P(x) \\ P(x) \wedge \top \wedge y = x + 1 &\rightarrow P(y) \\ P(x) \wedge x \geq 2 &\rightarrow \perp \end{aligned}$$

This transformation was also used to detect the `GOLEM` bug described in Sect. 3.

**PLUGBOTTOM**RHS. Given an UNSAT instance, this transformation chooses a clause  $\varphi \wedge p_1 \wedge \dots \wedge p_n \rightarrow h$  and replaces  $h$  with  $\perp$ . We, therefore, obtain

$$\varphi \wedge p_1 \wedge \dots \wedge p_n \rightarrow \perp.$$



The intuition here is similar to that for the `PLUGTOPLHS` transformation. In particular, we strengthen the RHS of an implication  $(A \rightarrow B) \equiv (\neg A \vee B)$  by replacing  $B$  with *false*. So, if the original disjunction does not hold, the transformed disjunction should also not hold.

As an example, consider again the unsatisfiable instance from Sect. 2 (also shown above). When replacing the RHS of the second clause, i.e.,  $P(y)$ , with  $\perp$ , we obtain:

$$\begin{aligned} x &\leq 0 \rightarrow P(x) \\ P(x) \wedge x < 2 \wedge y = x + 1 &\rightarrow \perp \\ P(x) \wedge x \geq 2 &\rightarrow \perp \end{aligned}$$

**DROPREDUNDANTCLAUSE.** Given an UNSAT instance, this transformation chooses a clause that does not appear in the refutation proof and removes it.

If a clause does not appear in the refutation proof, then it is not necessary to prove that the set of CHCs is unsatisfiable. Consequently, removing it from the set should not affect the solver response—it should still be UNSAT. Note that, instead of simply removing it, we could also apply any transformation to a clause not appearing in the refutation proof and still ensure that the transformed instance is UNSAT.

As an example, consider the clauses:

$$\begin{aligned} x < y &\rightarrow P(x, y) & (5) \\ P(x, 1) &\rightarrow Q & (6) \\ Q \wedge P(x, 7) \wedge x > y &\rightarrow R(y) & (7) \\ y < 0 \wedge P(x, y) &\rightarrow R(y) & (8) \\ R(3) &\rightarrow \perp & (9) \end{aligned}$$

A refutation proof for this instance is:

$$\left. \begin{array}{l} \top \xrightarrow{(5)} P(0, 1) \xrightarrow{(6)} Q \\ \top \xrightarrow{(5)} P(4, 7) \end{array} \right\} \xrightarrow{(7)} R(3) \xrightarrow{(9)} \perp$$

It says that, if  $x = 0, y = 1$ , we know that  $P(0, 1)$  holds from clause (5). The same clause also establishes  $P(4, 7)$  if  $x = 4, y = 7$ . Given  $P(0, 1)$ , the left-hand side of clause (6) holds, and thus, it establishes  $Q$  on the right-hand side. Given  $Q$  and  $P(4, 7)$ , if  $y = 3$ , the left-hand side of clause (7) holds, and as a result,  $R(3)$  holds. Finally, given  $R(3)$ , clause (9) derives  $\perp$ . Now, since clause (8) is not used in the refutation proof, applying the above transformation results in retaining only the other clauses:

$$\begin{aligned} x < y &\rightarrow P(x, y) \\ P(x, 1) &\rightarrow Q \\ Q \wedge P(x, 7) \wedge x > y &\rightarrow R(y) \\ R(3) &\rightarrow \perp \end{aligned}$$

As we briefly mentioned in Sect. 3, this transformation was used to detect the `GOLEM` bug of severity 1. Specifically, for the above transformed instance, `GOLEM` returned SAT instead of UNSAT.

**REPLACECLAUSEWITHFACT.** Given an UNSAT instance, this transformation selects a fact  $F = q(\dots)$  from the refutation proof, where  $q$  is an uninterpreted predicate. The transformation then replaces the clause that the refutation proof uses to derive  $F$  by  $\top \rightarrow F$ .

The intuition here is that any clauses that lead to the derivation of  $F$  are no longer needed. So, this transformation effectively eliminates the first part of the proof establishing  $F$ , states that  $F$

holds with clause  $\top \rightarrow F$ , and given  $F$ , it expects the solver to show that the transformed instance is also unsatisfiable.

As an example, consider again the unsatisfiable instance from Sect. 2 and its refutation proof. Let us select fact  $F = P(0)$  derived using clause (1). This transformation, therefore, replaces clause (1) by  $\top \rightarrow P(0)$  and generates the following (also unsatisfiable) instance:

$$\begin{aligned} & \top \rightarrow P(0) \\ & P(x) \wedge x < 2 \wedge y = x + 1 \rightarrow P(y) \\ & P(x) \wedge x \geq 2 \rightarrow \perp \end{aligned}$$

**Summary.** Transformations `DROPREDUNDANTCLAUSE` and `REPLACECLAUSEWITHFACT` transform an instance using both the solver response, i.e., that the original instance is found to be UNSAT, and the refutation proof. The remaining transformations rely only on the solver response.

`PLUGTOPLHS` weakens the left-hand side of an unsatisfiable clause, which as we explained, preserves its satisfiability. Similarly, `PLUGBOTTOMRHS` strengthens the right-hand side of an unsatisfiable clause. `DROPREDUNDANTCLAUSE` removes a clause that is not used in the refutation proof, whereas `REPLACECLAUSEWITHFACT` replaces a clause that is used to derive a fact from the proof with the fact.

### 4.3 Satisfiability-Preserving Transformations

The following transformation preserves the satisfiability of any instance, that is, satisfiable, resp. unsatisfiable, instances should remain satisfiable, resp. unsatisfiable.

**UNPLUGLHS+CLAUSE.** This transformation is similar to `UNPLUGLHS` except that it may also be applied to unsatisfiable instances. In comparison to `UNPLUGLHS`, it additionally introduces a clause  $\varphi_j \rightarrow p_{\text{fresh}}$ .

If the original instance is satisfiable, its model may be extended for the transformed instance with an interpretation of  $p_{\text{fresh}}$ , namely  $\varphi_j$ . Conversely, if the original instance is unsatisfiable, its refutation proof may be extended for the transformed instance with  $p_{\text{fresh}}$  facts.

For example, consider the unsatisfiable instance from Sect. 2 and its refutation proof. When replacing conjunct  $y = x + 1$  with  $p_{\text{fresh}}(x, y)$ , we obtain:

$$x \leq 0 \rightarrow P(x) \tag{10}$$

$$P(x) \wedge x < 2 \wedge p_{\text{fresh}}(x, y) \rightarrow P(y) \tag{11}$$

$$P(x) \wedge x \geq 2 \rightarrow \perp \tag{12}$$

$$y = x + 1 \rightarrow p_{\text{fresh}}(x, y) \tag{13}$$

The refutation proof now becomes:

$$\left. \begin{array}{l} \top \xrightarrow{(10)} P(0) \\ \top \xrightarrow{(13)} p_{\text{fresh}}(0, 1) \\ \top \xrightarrow{(13)} p_{\text{fresh}}(1, 2) \end{array} \right\} \xrightarrow{(11)} P(1) \left\} \xrightarrow{(11)} P(2) \xrightarrow{(12)} \perp$$

### 4.4 Instance-Fusion Transformations

These transformations fuse two instances into one, and the satisfiability of the newly generated instance depends on the satisfiability of the original instances.

**FUSESTRONG.** This transformation fuses two CHC instances as follows. Assume  $C_1$  and  $C_2$  are the clauses in each instance. The fused instance contains clauses  $C_1 \cup C_2$ . We guarantee (by

renaming) that the predicate names in  $C_1$  and  $C_2$  are disjoint. By conjoining the two sets of clauses, we ensure that the fused instance is satisfiable if both original instances are satisfiable, otherwise it is unsatisfiable.

As an example, consider clauses  $C_1$  to be

$$\begin{aligned} x \leq 5 \wedge P(x, y) \wedge Q(y) &\rightarrow R(x) \\ x = 0 \wedge R(0) &\rightarrow \perp, \end{aligned}$$

which are satisfiable, and clauses  $C_2$  (from Sect. 3) to be

$$\begin{aligned} x < y &\rightarrow P(x, y) \\ P(x, 1) &\rightarrow Q \\ Q \wedge P(x, 7) \wedge x > y &\rightarrow R(y) \\ R(3) &\rightarrow \perp, \end{aligned}$$

which are unsatisfiable. The fused instance is then

$$\begin{aligned} x \leq 5 \wedge P(x, y) \wedge Q(y) &\rightarrow R(x) \\ x = 0 \wedge R(0) &\rightarrow \perp \\ x < y &\rightarrow P'(x, y) \\ P'(x, 1) &\rightarrow Q' \\ Q' \wedge P'(x, 7) \wedge x > y &\rightarrow R'(y) \\ R'(3) &\rightarrow \perp, \end{aligned}$$

and it is unsatisfiable.

**FUSEWEAK.** This transformation fuses two CHC instances as follows. Assume  $C_1$  and  $C_2$  are the clauses in each instance. The transformation replaces all clauses of the form  $\varphi \wedge p_1 \wedge \dots \wedge p_n \rightarrow \perp$  with  $\varphi \wedge p_1 \wedge \dots \wedge p_n \rightarrow f_1$  in  $C_1$  and with  $\varphi \wedge p_1 \wedge \dots \wedge p_n \rightarrow f_2$  in  $C_2$ , where  $f_1$  and  $f_2$  are fresh uninterpreted predicates. It then adds the clause  $f_1 \wedge f_2 \rightarrow \perp$  expressing that if both instances are unsatisfiable, the fused instance is also unsatisfiable, otherwise it is satisfiable. Again, we guarantee (by renaming) that the predicate names in  $C_1$  and  $C_2$  are disjoint.

As an example, consider clauses  $C_1$  and  $C_2$  from the above transformation. The fused instance is then

$$\begin{aligned} x \leq 5 \wedge P(x, y) \wedge Q(y) &\rightarrow R(x) \\ x = 0 \wedge R(0) &\rightarrow F_1 \\ x < y &\rightarrow P'(x, y) \\ P'(x, 1) &\rightarrow Q' \\ Q' \wedge P'(x, 7) \wedge x > y &\rightarrow R(y) \\ R'(3) &\rightarrow F_2 \\ F_1 \wedge F_2 &\rightarrow \perp, \end{aligned}$$

and it is satisfiable.

#### 4.5 Transformation Combinations

Of course, transformations may be stacked, that is, we can apply multiple transformations to a given instance at a time as long as they agree on the expected solver response for the transformed instance.

Within this constraint, the selection and application order of transformations is randomized to maximize diversity.

More interestingly, however, HORNGATOR can combine certain transformations in a more meaningful way. In particular, regarding the UNPLUGLHS and UNPLUGLHS+CLAUSE transformations, if  $\varphi_j$  occurs in constraint  $\varphi$  of more than one clause in the instance, we can replace it multiple times using the *same* uninterpreted predicate  $p_{fresh}$ . Moreover, if  $\neg\varphi_j$  occurs in clauses where  $h \equiv \perp$ , we can also replace it with the same  $p_{fresh}$  appearing on the right-hand side (see UNPLUGRHS).

## 5 Experimental Evaluation

We evaluate HORNGATOR by testing five CHC solvers that participate in CHC-COMP, namely ELDARICA, GOLEM, SPACER, THETA, and ULTIMATE TREE AUTOMIZER. In our evaluation, we address the following research questions:

**RQ1:** How effective is HORNGATOR in detecting bugs of different severity in CHC solvers?

**RQ2:** What are characteristics of the detected bugs?

**RQ3:** How efficient is HORNGATOR?

**RQ4:** Which transformations were effective in bug detection?

**RQ5:** How effective is interrogation testing vs. metamorphic testing?

**RQ6:** How does HORNGATOR compare to HORNFUZZ?

### 5.1 Experimental Setup

**Testing time.** We implemented an early version of HORNGATOR and started testing ELDARICA and SPACER. We then improved our tool, designed more transformations, and added support for more solvers. Each solver was tested between one and six months. During the testing time, we did not report any duplicate bugs. Whenever a bug was detected, we either paused testing the affected solver until a fix was provided by the developers, or we manually de-duplicated any additional bugs before reporting them.

**Fuzzing campaigns.** We define two setups for our fuzzing campaigns, namely the *bug-detection* and *time-to-bug* setups.

The bug-detection setup was used to find previously unknown bugs in the solvers under test during the above testing time. In this setup, the interrogator initializes the knowledge base with five random seed instances from CHC-COMP. We use all 2,336 instances from the 2023 edition of the competition as well as all 300 instances from the newly introduced category of “ADTs with arrays” in the 2024 edition. The knowledge base is reset after 100 queries to the solver, and following each reset, the interrogator uses new random seed instances to re-initialize the knowledge base. We use a solver timeout of 60s per instance.

The time-to-bug setup is used to measure the time it takes HORNGATOR to re-find fixed bugs. For this setup, the interrogator initializes the knowledge base only with those seed instances from CHC-COMP that were previously needed to expose the bug. Note that these instances are at most two—all our transformations are applied to a single instance, except for the fusion ones, which are applied to two. The knowledge base is again reset after 100 queries to the solver, and following each reset, the interrogator uses the same seed instances to re-initialize the knowledge base. We again use a solver timeout of 60s per instance and introduce a timeout of 72h for the fuzzing campaign. To ensure reproducibility of the results, each fuzzing campaign is controlled by a numeric random seed. To account for the randomness in HORNGATOR, we run our time-to-bug experiments with 20 different numeric random seeds.

**Hardware.** We performed all experiments on a machine with two AMD EPYC 9474F CPUs @ 3.60GHz and 1.5TB of memory, running Debian GNU/Linux 12 (bookworm).

Tab. 2. Bugs detected by HORN<sub>G</sub>ATOR.

| Bug ID | CHC Solver      | Bug Severity |   |    |    | Bug Status |
|--------|-----------------|--------------|---|----|----|------------|
|        |                 | 1            | 2 | 3a | 3b |            |
| 1      | ELDARICA        |              | ✓ |    |    | Fixed      |
| 2      | ELDARICA        |              |   | ✓  |    | Fixed      |
| 3      | ELDARICA        |              | ✓ |    |    | Fixed      |
| 4      | ELDARICA        |              |   | ✓  |    | Fixed      |
| 5      | ELDARICA        |              | ✓ |    |    | Fixed      |
| 6      | ELDARICA        |              | ✓ |    |    | Fixed      |
| 7      | GOLEM           |              | ✓ |    |    | Fixed      |
| 8      | GOLEM           | ✓            |   |    |    | Fixed      |
| 9      | GOLEM           | ✓            |   |    |    | Fixed      |
| 10     | GOLEM           |              | ✓ | ✓  | ✓  | Fixed      |
| 11     | GOLEM           |              | ✓ |    |    | Fixed      |
| 12     | SPACER          |              | ✓ | ✓  |    | Fixed      |
| 13     | SPACER          | ✓            | ✓ |    |    | Fixed      |
| 14     | SPACER          | ✓            | ✓ |    |    | Fixed      |
| 15     | SPACER          | ✓            | ✓ |    |    | Fixed      |
| 16     | SPACER          | ✓            |   |    |    | Fixed      |
| 17     | THETA           |              | ✓ |    |    | Fixed      |
| 18     | U. T. AUTOMIZER |              | ✓ |    |    | Fixed      |
| A      | ELDARICA        | ✓            |   |    |    | Confirmed  |
| B      | SPACER          |              |   | ✓  |    | Confirmed  |
| C      | SPACER          | ✓            |   |    |    | Confirmed  |

## 5.2 Experimental Results

We now discuss our findings for each research question.

**RQ1: Effectiveness of HORN<sub>G</sub>ATOR.** Tab. 2 shows the list of unique bugs detected by HORN<sub>G</sub>ATOR using the bug-detection setup. The first column provides an identifier for each bug and links to the (anonymized) bug report on GitHub. The second column shows the CHC solver for which the bug was found. The next two columns indicate the bug severity (as explained in Tab. 1) and its status (confirmed or fixed). We use numbers to identify fixed bugs and letters for confirmed ones.

As shown in the table, a bug may have different severity classifications. This is because there are multiple ways in which it might affect the solver, and therefore manifest itself. Bug severity is determined through manual inspection, and of course, by confirming with the solver developers. In the table, we show all the ways in which each bug manifested itself during our experiments. For example, due to bug 10, GOLEM erroneously returned UNSAT for the satisfiable instance of Fig. 2b, but there were also cases where this bug caused the solver to return an invalid model (severity 3a) or refutation proof (severity 3b).

The lower bug count in THETA and U. T. AUTOMIZER is primarily due to two factors. First, these solvers are significantly slower than others, which reduces test throughput and limits the number of explored inputs during the fuzzing campaign. Second, they support fewer theories compared to the other solvers, which restricts the number of suitable seed input files. Additionally, during our testing period, the issues in these solvers took a long time to be acknowledged and fixed, so we were unable to continue fuzzing them effectively—we repeatedly encountered the same bug without further progress.

$$\begin{array}{ll}
x < y \rightarrow P(x, y) & (x \wedge y) \wedge \neg x \wedge P(1) \rightarrow P(4) \\
x < y \wedge P(y, z) \rightarrow Q(x, z) & P(2) \rightarrow \perp \\
Q(1, 3) \rightarrow R_1 & \text{(b) Bug 10 in GOLEM.} \\
Q(0, 2) \rightarrow R_2 & \top \rightarrow P(0, x, y, y) \\
R_1 \wedge R_2 \rightarrow \perp & (\forall (e \ E).(e = c)) \wedge \neg(x = 0 \wedge u = z) \wedge P(x, y, z, u) \rightarrow \perp \\
\text{(a) Bug 8 in GOLEM.} & \text{(c) Bug 1 in ELDARICA.}
\end{array}$$

Fig. 2. Bugs found by HORNGATOR in GOLEM and ELDARICA.

In total, HORNGATOR detected 21 unique bugs, all of which are confirmed by the developers and 18 are fixed. Considering only the highest severity of each bug, HORNGATOR detected eight bugs of severity 1, ten bugs of severity 2, and three bugs of severity 3a. We found bugs in all solvers we tested and of all severity classifications we considered.

**RQ2: Detected bugs.** Here, we discuss sample bugs found by HORNGATOR in GOLEM, ELDARICA, and SPACER, which were fixed by the developers.

Fig. 2a shows an instance that revealed bug 8 of severity 1 in GOLEM. The instance is unsatisfiable because the first two clauses establish  $Q(1, 3)$  and  $Q(0, 2)$ . Nevertheless, GOLEM returned SAT. The bug was due to incorrect handling of variable names—upon parsing, auxiliary variables are prefixed with “aux”, but this was not done when such variables were added during preprocessing. Interestingly, the buggy code contained the comment “Fixme: This is hackish”. This bug could only be revealed with our DROPREDUNDANTCLAUSE and REPLACECLAUSEWITHFACT transformations.

Fig. 2b shows an instance that revealed bug 10 of severity 2 in GOLEM. The instance is trivially satisfiable since the left-hand side of the first clause evaluates to false. This bug was found in the solver’s preprocessing component, which incorrectly classified the instance as trivially unsatisfiable.

Fig. 2c shows an instance that revealed bug 1 of severity 2 in ELDARICA. It uses algebraic data types (ADTs) to define an enum  $E$  (not shown in the figure), which only has value  $c$ . Given this,  $\forall (e \ E).(e = c)$  on the left-hand side of the last clause is a tautology. SPACER returned UNSAT for this satisfiable instance due to a bug related to quantifier elimination in the context of ADTs. This bug could only be revealed with our PLUGMODELLHS transformation. In particular, formula  $\forall (e \ E).(e = c)$  is part of a model returned by the solver. Note that plugging in the entire model, i.e., also instantiating  $P$ , does not reveal the bug.

Bug 12 of severity 2 in SPACER was found when the solver returned UNSAT for a satisfiable instance. The bug was due to SPACER’s slicing procedure, which aggressively sliced away relevant parts of the instance.

Overall, we received very positive developer feedback for our bug reports. We already mentioned in Sect. 3 that the GOLEM developers encouraged us to report more bugs. The ELDARICA developers called bugs 2 and 3 a “great catch”, and for bug 4, they said: “The bug must have been in the solver since around 2010, so this is an awesome catch.”

**RQ3: Efficiency of HORNGATOR.** To evaluate our tool’s efficiency, we measure the time it takes HORNGATOR to re-find the bugs it previously detected. For this experiment, we use the time-to-bug setup and include only fixed bugs to ensure that a detected bug is indeed the one we intended to re-find. In particular, for each detected bug, we automatically apply the fix proposed by the solver developers and check that the bug disappears.

We exclude bugs 3 and 4 from this experiment as HORNGATOR cannot re-find them within the time limit of 72h. These bugs are very intricate to detect and were found after weeks of testing

Tab. 3. Average time for HORN GATOR and its baselines to detect a fixed bug.

| Bug ID | HORN GATOR         |                    |                    |                    |
|--------|--------------------|--------------------|--------------------|--------------------|
|        | w/o kb&wtn         | w/o kb             | w/o wtn            | full               |
| 1      | –                  | 02h 56m 40s        | –                  | <b>00h 28m 27s</b> |
| 2      | 16h 10m 57s        | 17h 30m 04s        | 06h 50m 30s        | <b>06h 39m 12s</b> |
| 5      | 33h 31m 59s        | 13h 21m 29s        | 12h 40m 00s        | <b>02h 12m 16s</b> |
| 6      | 01h 09m 08s        | 01h 37m 19s        | 00h 07m 59s        | <b>00h 07m 02s</b> |
| 8      | –                  | <b>00h 00m 51s</b> | –                  | 00h 01m 05s        |
| 9      | –                  | 00h 02m 48s        | –                  | <b>00h 01m 49s</b> |
| 10     | 00h 04m 42s        | <b>00h 01m 00s</b> | 00h 14m 49s        | 00h 07m 47s        |
| 11     | –                  | –                  | <b>07h 17m 44s</b> | 12h 09m 56s        |
| 12     | 02h 58m 33s        | <b>01h 10m 29s</b> | 02h 04m 02s        | 01h 58m 22s        |
| 13     | –                  | –                  | 24h 25m 48s        | <b>12h 31m 41s</b> |
| 14     | –                  | –                  | <b>01h 42m 59s</b> | 02h 11m 38s        |
| 15     | 00h 20m 26s        | 00h 17m 05s        | 00h 21m 49s        | <b>00h 15m 10s</b> |
| 16     | 02h 14m 27s        | 02h 43m 55s        | <b>00h 56m 47s</b> | 00h 59m 37s        |
| HF5    | 00h 00m 14s        | 00h 00m 23s        | <b>00h 00m 08s</b> | 00h 00m 11s        |
| HF8    | <b>00h 00m 31s</b> | 00h 00m 42s        | 00h 00m 52s        | 00h 00m 58s        |
| HF9    | 06h 16m 31s        | 01h 07m 23s        | 00h 14m 21s        | <b>00h 09m 03s</b> |

ELDARICA; in fact, they are triggered in ELDARICA’s underlying SMT solver, PRINCESS. We also exclude bugs 7 and 18. For these bugs, the solver returned an incorrect response because, as the developers informed us, the instances were beyond the scope of GOLEM and ULTIMATE TREE AUTOMIZER, respectively, e.g., by using a quantified constraint  $\varphi$ . Since extending the solvers to handle such instances is a major undertaking for the developers, they implemented a temporary fix by returning an error (instead of the incorrect response). Finally, we exclude bug 17, which was found in an abstract domain of THETA. The developers have currently disabled the domain and opened a new issue to revisit its soundness in the future.

We also measure how long it takes HORN GATOR to detect fixed bugs found by HORN FUZZ [31]. To distinguish them from our bugs, we label the HORN FUZZ bugs using “HF” and a number; as usual, each label links to the corresponding bug report. HORN FUZZ detected a total of eleven fixed bugs; one of these turned out to be a duplicate, leaving us with ten. We exclude HF1 because it cannot be detected by HORN GATOR (see RQ6 for more details). We also exclude HF2 because the fix was to disable the buggy solver option. Finally, we exclude HF3, HF4, HF6, HF7, and HF10 since no fuzzing is necessary to detect them. Specifically, the bugs are immediately detected by simply plugging in the returned model into certain existing CHC-COMP instances.

The first and last columns of Tab. 3 show the average time (across 20 fuzzing campaigns with different numeric random seeds) it takes HORN GATOR to re-find the bugs discussed above. (The other columns are discussed in RQ5.) The average time to bug varies from 11s (for bug HF5 of severity 3a) to over 12h (for bug 13 of severity 1 and 2). Unsurprisingly, most of this time is spent on running the solver under test; in fact, for all bugs, this time is  $\geq 85\%$ , and for all but two bugs (8 and 13), this time is  $\geq 95\%$ .

**RQ4: Transformations.** When running the above experiment, we also inspected the sequences of transformations that led to revealing each bug. The transformations appearing most frequently in the sequences are ADDCSTR LHS (28%), UNPLUG LHS+CLAUSE (14%), PLUG TOP LHS (14%), and FUSE WEAK (14%). Transformations DROP REDUNDANT CLAUSE (1%) and REPLACE CLAUSE WITH FACT

(2%) are the least frequent since they may only be applied to UNSAT instances for which a refutation proof is generated—refutation proofs cannot always be generated by certain solvers.

However, even these least frequent transformations were critical for bug detection. In particular, they appear in 3% and 9% of the sequences as the last transformation that revealed the bug. For example, bug 9 was only detected due to either `DROPREDUNDANTCLAUSE` or `REPLACECLAUSEWITHFACT` in all 20 runs.

*All transformations were effective either by contributing new instances to the knowledge base that eventually led to a bug, or by being applied last and immediately triggering a bug.*

Note that we do not claim that every transformation is strictly necessary for every bug. Computing a minimal set of transformations is computationally impractical: with 12 transformations, this would require exploring up to  $2^{12}$  configurations. Moreover, removing a transformation that appears unnecessary for the currently known bugs might reduce diversity and prevent `HORNGATOR` from uncovering other, as-yet-unknown bugs in future campaigns. Instead, RQ5 demonstrates that certain groups of transformations—particularly those leveraging models and proofs—are critical for `HORNGATOR`'s overall effectiveness.

**RQ5: Interrogation vs. metamorphic testing.** Interrogation testing extends metamorphic testing in two key ways: it leverages analysis witnesses and maintains a knowledge base. So, in addition to the solver response, `HORNGATOR` uses generated witnesses—models or refutation proofs—to construct new CHC instances. It also integrates a knowledge base that accumulates diverse queries over time. To assess the effectiveness of these interrogation-testing extensions in the CHC setting, we introduce the following baselines.

**HORNGATOR w/o kb:** This baseline has the same functionality as `HORNGATOR`, but it does not use a knowledge base. Specifically, it always applies transformations to the instance that was generated last, and any previous instances are discarded. The `FUSEWEAK` and `FUSESTRONG` transformations cannot be applied as they require two instances as input.

**HORNGATOR w/o wtn:** This baseline has the same functionality as `HORNGATOR`, but it disables all transformations that use a witness, i.e., `PLUGMODEL LHS`, `PLUGMODEL RHS`, `DROPREDUNDANT-CLAUSE`, and `REPLACECLAUSEWITHFACT`.

**HORNGATOR w/o kb&wtn:** This baseline does not use a knowledge base and disables all transformations that use a witness; it is most similar to `HORNFUZZ`.

Note that the last two baselines are still able to detect bugs of severity 3a, where the model of a satisfiable instance is invalid. This is achieved by a model-validation step performed for all instances that are found SAT: it consists of plugging the entire model into the CHC instance—essentially converting it into a regular SMT instance—and checking its satisfiability.

The second, third, and fourth columns of Tab. 3 show the time to bug for these baselines. “–” indicates that none of the 20 fuzzing campaigns found the bug within 72h. The remaining bugs were found in all 20 campaigns, with the exception of bug 5, which was only found in 11 campaigns for `HORNGATOR w/o kb&wtn`; the time is averaged across these 11 campaigns and is shown in italics in the table. For each bug, the lowest average time is shown in bold.

As shown in the table, `HORNGATOR` is the only configuration that finds all bugs and is faster than the baselines for eight of them. In contrast, `HORNGATOR w/o kb&wtn` fails to detect six bugs, four of which have severity 1, and only detects bug 5 in eleven out of 20 campaigns. All remaining bugs are found faster by `HORNGATOR` with the exception of bugs 10 and HF8. For bug 10, the slowdown of 1.66x is related to the use of fusion transformations as described in the next paragraph. Bug HF8 is found in less than a minute by all configurations.

In comparison to `HORNGATOR`, the baseline w/o kb fails to find three bugs, two of which have severity 1. For nine of the remaining bugs, `HORNGATOR` achieves a speedup between 1.13x (bug 15)



and 13.87x (bug 6), whereas for the other four bugs, HORNGATOR is slower between 1.27x (bug 8) and 7.78x (bug 10). The latter slowdown is significant, yet a clear outlier in the table. Upon investigation, we found that the slowdown is caused by the fusion transformations, which generate large instances that might take longer to solve. However, as we discussed in the previous RQ, these transformations are critical in detecting bugs; they constitute 24% of the transformations in all bug-revealing sequences and appear in 33% of the sequences as the last transformation that revealed the bug.

Finally, the baseline w/o wtn fails to find three bugs, two of which have severity 1. For eight of the remaining bugs, HORNGATOR achieves a speedup between 1.03x (bug 2) and 5.75x (bug 5), whereas for the other five bugs, HORNGATOR is slower between 1.05x (bug 16) and 1.67x (bug 11). Although small, such a slowdown may occur for bugs that do not require any witness transformations to be detected. In these cases, applying them simply increases the time to bug.

*Overall, each extension of interrogation testing over metamorphic testing independently contributes to finding more bugs, and in most cases, faster. Using both constitutes the best configuration.*

**RQ6: Comparison to HORNFUZZ.** HORNFUZZ uses metamorphic testing to test SPACER; as mentioned earlier, its design most closely resembles baseline HORNGATOR w/o kb&wtn. In this RQ, we first discuss the fixed bugs found by HORNFUZZ. We then check if our bugs existed in the version of SPACER that HORNFUZZ tested. Last, we try to use HORNFUZZ to detect our bugs.

HORNFUZZ detected bugs HF1 and HF2 of severity 2. HF1 was found by a transformation that replaces a conjunct  $\varphi$  on the LHS of a clause with  $\varphi \wedge \varphi$ . By iteratively applying this transformation, HORNFUZZ built an instance with a clause containing over 1M conjuncts, which exceeded an internal limit of SPACER. Since HORNGATOR does not duplicate conjuncts, it cannot trigger this bug. Even though the developers provided a fix, they said that this behavior is unlikely to be exposed in practice. HF2 uses a buggy option that was used for experiments a decade ago; the option is now removed.

HORNFUZZ also found bugs HF3, HF4, HF5, HF6, HF7, HF8, HF9, and HF10 of severity 3a. Recall that, even though HORNFUZZ does not use witnesses in its metamorphic transformations, it detects bugs of this severity with a model-validation check, i.e., by plugging the entire model into a satisfiable CHC instance. HORNGATOR can detect all these bugs, typically in less than a minute. In our experiments, we noticed that five of them can be triggered by simply applying the model-validation check for existing satisfiable CHC-COMP instances; no fuzzing is necessary when using such instances as seeds. The remaining three bugs (HF5, HF8 and HF9) are among the ones with the shortest time to bug in Tab. 3.

Next, we discuss whether HORNFUZZ could have found our bugs but did not. To this end, we check whether they existed in SPACER in May'23, when HORNFUZZ was used to test the solver. HORNGATOR detected seven SPACER bugs, namely 12, 13, 14, 15, 16, B, and C. Of these, four were already present in the old SPACER version and missed by HORNFUZZ, namely 12, 13, 14, and B.

Finally, we ran HORNFUZZ with the seed instances that revealed our 5 fixed SPACER bugs, namely 12, 13, 14, 15, and 16, and checked whether it can detect them. Bug 14 cannot be found by HORNFUZZ by construction as it relies on two CHC-COMP seeds. As for the remaining four bugs, HORNFUZZ could not detect them within the time limit of 72h that we used for our experiments.

*Based on these findings, HORNGATOR is significantly more effective in bug finding than HORNFUZZ.*

### 5.3 Threats to Validity

The validity of our results depends on the CHC solvers under test, the seed instances, and the randomness in HORNGATOR and its baselines. To limit the effect of these threats, we test five solvers that participate in CHC-COMP, use thousands of seed instances from CHC-COMP, and run our fuzzing campaigns with 20 numeric random seeds.

## 6 Related Work

**HORNFUZZ.** The most closely related work is HORNFUZZ [31], which proposes a fuzzing technique for CHC solvers based on metamorphic testing [12]. In contrast, HORN GATOR is based on interrogation testing [18], an extension of metamorphic testing that was already used for testing program analyzers. It introduces two key innovations: (1) using more detailed information—such as witnesses—to design the transformations, and (2) using the knowledge base to form new (trick) queries by building on past results stored there. RQ6 compares the two fuzzers in terms of their bug-finding effectiveness.

**SMT solvers.** Several fuzzers have been proposed for SAT solvers (e.g., [8]), string solvers (e.g., [5, 9]), and SMT solvers (e.g., [6, 7, 20, 24, 29, 30, 34–38]). We refer the reader to Sect. 1 for a detailed discussion of why SMT fuzzers are not well suited for CHC solvers.

In brief, SMT fuzzers generate or mutate inputs tailored to the subset of SMT-LIB accepted by SMT solvers, which differs substantially from the structurally constrained input format required by CHC solvers. For example, DIVER [20] applies unrestricted random mutations, and YINYANG [36] introduces fresh free variables—both approaches that violate CHC syntax. Grammar-based generators such as ET [34] and FALCON [37] also miss CHC-specific constructs (e.g., uninterpreted predicates) and thus fail to trigger CHC-specific bugs.

SMT fuzzers also tend to focus on refutational soundness bugs—cases where a satisfiable input is incorrectly reported as UNSAT—which are considered the most severe in the SMT setting. However, in the CHC domain, the most critical bugs (severity 1) occur when SAT is returned for an unsatisfiable instance. SMT fuzzers like DIVER [20] and STORM [24], which only generate satisfiable inputs, are incapable of uncovering such errors.

Finally, SMT fuzzers that rely on differential testing, such as OpFUZZ[35], TYPEFUZZ[30], FALCON [37], and ET [34], inherit the limitations discussed in Sect. 1, including theory mismatches, solver-specific constructs, and ambiguity when solvers disagree.

**Datalog engines.** CHC solvers are also connected to Datalog engines given that a Datalog constraint can be viewed as a specific type of Horn clause. Specifically, Datalog typically considers only predicates over finite domains instead of infinite domains, such as mathematical integers. As a result, CHC solvers are, in theory, more expressive than Datalog engines. There are three existing fuzzers for Datalog engines, namely, QUERYFUZZ [23] and DLSMITH [25], which are based on metamorphic testing, as well as DEOPT [40], which is based on incremental rule evaluation.

Taking inspiration from our CHC transformations, Datalog engines also expose interesting additional information that could be used to design transformations for these engines. For instance, some Datalog engines expose provenance information to explain which rule applications were used to derive a given fact; this shares some similarity with refutation proofs, which explain how a solver derived a contradiction. None of the existing fuzzers rely on such detailed information.

**Program analyzers.** Beyond constraint solvers, there is a substantial body of work on testing more general-purpose program-analysis infrastructure—such as, abstract interpreters [10, 21, 28, 32], model checkers [21, 39], and symbolic execution engines [19]—and compilers [11].

## 7 Conclusion

We presented the first interrogation-testing technique for CHC solvers and implemented it in HORN GATOR. Using models, refutation proofs, and a knowledge base, HORN GATOR forms diverse CHC instances that are able to trigger bugs of severity 1–3. In fact, HORN GATOR detected 21 unique bugs in five solvers, 18 of which have severity 1 or 2. We also showed that HORN GATOR is significantly more effective in bug finding than related work.

In future work, we plan to further increase the effectiveness of HORN<sub>GATOR</sub> by devising a diversity criterion that filters the instances that are added to the knowledge base, exploring how to better select instances to fuzz from the knowledge base (instead of doing it uniformly at random), and developing a power schedule that assigns more energy to “interesting” instances (i.e., fuzzes them more).

## Data Availability

Our tool is publicly available at <https://github.com/Rigorous-Software-Engineering/HornGator> and archived at <https://doi.org/10.5281/zenodo.19132854>.

## Acknowledgments

We thank the CHC-solver developers for their valuable work and assistance. This work was supported by Maria Christakis’ ERC Starting grant 101076510.

## References

- [1] [n. d.]. CHC-COMP. <https://chc-comp.github.io>.
- [2] [n. d.]. The Satisfiability Modulo Theories Library. <http://smtlib.cs.uiowa.edu>.
- [3] Nikolaj S. Bjørner, Arie Gurfinkel, Kenneth L. McMillan, and Andrey Rybalchenko. 2015. Horn Clause Solvers for Program Verification. In *Fields of Logic and Computation II (LNCS, Vol. 9300)*. Springer, 24–51.
- [4] Martin Blicha, Konstantin Britikov, and Natasha Sharygina. 2023. The Golem Horn Solver. In *CAV (LNCS, Vol. 13965)*. Springer, 209–223.
- [5] Dmitry Blotsky, Federico Mora, Murphy Berzish, Yunhui Zheng, Ifaz Kabir, and Vijay Ganesh. 2018. StringFuzz: A Fuzzer for String Solvers. In *CAV (LNCS, Vol. 10982)*. Springer, 45–51.
- [6] Mauro Bringolf, Dominik Winterer, and Zhendong Su. 2022. Finding and Understanding Incompleteness Bugs in SMT Solvers. In *ASE*. ACM, 43:1–43:10.
- [7] Robert Brummayer and Armin Biere. 2009. Fuzzing and Delta-Debugging SMT Solvers. In *SMT*. ACM, 1–5.
- [8] Robert Brummayer, Florian Lonsing, and Armin Biere. 2010. Automated Testing and Debugging of SAT and QBF Solvers. In *SAT (LNCS, Vol. 6175)*. Springer, 44–57.
- [9] Alexandra Bugariu and Peter Müller. 2020. Automatically Testing String Solvers. In *ICSE*. ACM, 1459–1470.
- [10] Alexandra Bugariu, Valentin Wüstholtz, Maria Christakis, and Peter Müller. 2018. Automatically Testing Implementations of Numerical Abstract Domains. In *ASE*. ACM, 768–778.
- [11] Junjie Chen, Jibesh Patra, Michael Pradel, Yingfei Xiong, Hongyu Zhang, Dan Hao, and Lu Zhang. 2020. A Survey of Compiler Testing. *Comput. Surv.* 53 (2020), 4:1–4:36. Issue 1.
- [12] Tsong Yueh Chen, S. C. Cheung, and Siu-Ming Yiu. 1998. *Metamorphic Testing: A New Approach for Generating Next Test Cases*. Technical Report HKUST-CS98-01. HKUST.
- [13] Daniel Dietsch, Matthias Heizmann, Jochen Hoenicke, Alexander Nutz, and Andreas Podolski. 2019. Ultimate TreeAutomizer (CHC-COMP Tool Description). In *HCVS/PERR@ETAPS (EPTCS, Vol. 296)*. 42–47.
- [14] Arie Gurfinkel and Nikolaj S. Bjørner. 2019. The Science, Art, and Magic of Constrained Horn Clauses. In *SYNASC*. IEEE Computer Society, 6–10.
- [15] Arie Gurfinkel, Temesghen Kahsai, Anvesh Komuravelli, and Jorge A. Navas. 2015. The SeaHorn Verification Framework. In *CAV (LNCS, Vol. 9206)*. Springer, 343–361.
- [16] Hossein Hojjat and Philipp Rümmer. 2018. The ELDARICA Horn Solver. In *FMCAD*. IEEE Computer Society, 1–7.
- [17] Temesghen Kahsai, Philipp Rümmer, Huascar Sanchez, and Martin Schäfer. 2016. JayHorn: A Framework for Verifying Java Programs. In *CAV (LNCS, Vol. 9779)*. Springer, 352–358.
- [18] David Kaindlstorfer, Anastasia Isychev, Valentin Wüstholtz, and Maria Christakis. 2024. Interrogation Testing of Program Analyzers for Soundness and Precision Issues. In *ASE*. ACM, 319–330.
- [19] Timotej Kapus and Cristian Cadar. 2017. Automatic Testing of Symbolic Execution Engines via Program Generation and Differential Testing. In *ASE*. IEEE Computer Society, 590–600.
- [20] Jongwook Kim, Sunbeom So, and Hakjoo Oh. 2023. Diver: Oracle-Guided SMT Solver Testing with Unrestricted Random Mutations. In *ICSE*. IEEE Computer Society, 2224–2236.
- [21] Christian Klinger, Maria Christakis, and Valentin Wüstholtz. 2019. Differentially Testing Soundness and Precision of Program Analyzers. In *ISSTA*. ACM, 239–250.
- [22] Anvesh Komuravelli, Arie Gurfinkel, and Sagar Chaki. 2014. SMT-Based Model Checking for Recursive Programs. In *CAV (LNCS, Vol. 8559)*. Springer, 17–34.

- [23] Muhammad Numair Mansur, Maria Christakis, and Valentin Wüstholtz. 2021. Metamorphic Testing of Datalog Engines. In *ESEC/FSE*. ACM, 639–650.
- [24] Muhammad Numair Mansur, Maria Christakis, Valentin Wüstholtz, and Fuyuan Zhang. 2020. Detecting Critical Bugs in SMT Solvers Using Blackbox Mutational Fuzzing. In *ESEC/FSE*. ACM, 701–712.
- [25] Muhammad Numair Mansur, Valentin Wüstholtz, and Maria Christakis. 2023. Dependency-Aware Metamorphic Testing of Datalog Engines. In *ISSTA*. ACM, 236–247.
- [26] Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. 2020. RustHorn: CHC-Based Verification for Rust Programs. In *ESOP (LNCS, Vol. 12075)*. Springer, 484–514.
- [27] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10 (1998), 100–107. Issue 1.
- [28] Jan Midtgaard and Anders Möller. 2017. QuickChecking Static Analysis Properties. *Softw. Test., Verif. Reliab.* 27 (2017). Issue 6.
- [29] Aina Niemetz, Mathias Preiner, and Armin Biere. 2017. Model-Based API Testing for SMT Solvers. In *SMT*. 10 pages.
- [30] Jiwon Park, Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2021. Generative Type-Aware Mutation for Testing SMT Solvers. *PACMPL* 5 (2021), 1–19. Issue OOPSLA.
- [31] Anzhela Sukhanova and Valentyn Sobol. 2023. HornFuzz: Fuzzing CHC solvers. In *EASE*. ACM, 83–92.
- [32] Jubi Taneja, Zhengyang Liu, and John Regehr. 2020. Testing Static Analyses for Precision and Soundness. In *CGO*. ACM, 81–93.
- [33] Tamás Tóth, Ákos Hajdu, András Vörös, Zoltán Micskei, and István Majzik. 2017. Theta: A Framework for Abstraction Refinement-Based Model Checking. In *FMCAD*. IEEE Computer Society, 176–179.
- [34] Dominik Winterer and Zhendong Su. 2024. Validating SMT Solvers for Correctness and Performance via Grammar-Based Enumeration. *PACMPL* 8 (2024), 2378–2401. Issue OOPSLA2.
- [35] Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2020. On the Unusual Effectiveness of Type-Aware Operator Mutations for Testing SMT Solvers. *PACMPL* 4 (2020), 193:1–193:25. Issue OOPSLA.
- [36] Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2020. Validating SMT Solvers via Semantic Fusion. In *PLDI*. ACM, 718–730.
- [37] Peisen Yao, Heqing Huang, Wensheng Tang, Qingkai Shi, Rongxin Wu, and Charles Zhang. 2021. Fuzzing SMT Solvers via Two-Dimensional Input Space Exploration. In *ISSTA*. ACM, 322–335.
- [38] Peisen Yao, Heqing Huang, Wensheng Tang, Qingkai Shi, Rongxin Wu, and Charles Zhang. 2021. Skeletal Approximation Enumeration for SMT Solver Testing. In *ESEC/FSE*. ACM, 1141–1153.
- [39] Chengyu Zhang, Ting Su, Yichen Yan, Fuyuan Zhang, Geguang Pu, and Zhendong Su. 2019. Finding and Understanding Bugs in Software Model Checkers. In *ESEC/FSE*. ACM, 763–773.
- [40] Chi Zhang, Linzhang Wang, and Manuel Rigger. 2024. Finding Cross-Rule Optimization Bugs in Datalog Engines. *PACMPL* 8 (2024), 110–136. Issue OOPSLA1.

Received 2025-09-10; accepted 2025-12-22